

Contents

Prayer Roster Backend — Phase 1: Architecture & Domain Design	1
Context	1
1. Recommended JHipster Configuration	2
2. Domain Model — Entities and Why	2
User (reuse JHipster's OAuth2 User, don't duplicate)	2
Role, Permission	3
WeeklyPrayerConfiguration (the recurring template)	3
PrayerSession (generated, actual dated data)	3
PrayerAssignment — the Timefold planning entity (key design decision)	3
UserAvailability	3
Roster vs RosterGeneration (the “evaluate both” ambiguity, resolved)	3
Notification, NotificationPreference, ReminderConfiguration, ReminderSent	4
3. Entity Relationships (cardinalities, constraints)	4
4–5. Weekly Configuration vs Generated Sessions	4
6. Timefold Planning Model	5
7. Hard Constraints (Constraint Streams, HardSoftScore)	5
8. Soft Constraints	6
9. Dynamic RBAC Design (ADMIN vs SUPER_ADMIN)	6
10. Google Authentication Architecture	6
Super Admin bootstrap	7
11. Roster Lifecycle	7
12. Rescheduling Architecture	7
13. Notification Architecture	7
14. API Design	8
15. Package/Module Structure (modular monolith)	8
Assumptions flagged for review (safe defaults chosen, not yet confirmed)	8
Next Steps	9

Prayer Roster Backend — Phase 1: Architecture & Domain Design

Context

This is a greenfield backend (backend/ is currently empty, no git repo yet). The goal is a production-ready JHipster + Spring Boot + Timefold Solver backend for a church's recurring weekly prayer program: a Super Admin configures the weekly pattern once, the system derives actual dated PrayerSessions for a planning period, Timefold assigns eligible users as moderator/preacher, and users are notified/reminded in French by default.

This document is **Phase 1 only** — architecture and domain design. No code has been generated yet; implementation starts in later phases after this is approved.

Three architecture forks were resolved with the project owner before writing this doc: - **Auth**: backend is a stateless **OAuth2 Resource Server** validating Google-issued ID tokens (JWT) as Bearer tokens. No server-side login redirect/session — a separate frontend/SPA performs the Google sign-in and forwards the ID token. - **Horizon**: the monthly cron maintains a **rolling 2-month-ahead** generated window (not 1), in addition to admin-triggered generation for any custom range. - **Publish gate**: generation (cron, manual, or reschedule) **auto-publishes** on success — no manual review step required for the happy path.

1. Recommended JHipster Configuration

- **JHipster 8.11.0 pinned exactly** (Spring Boot 3.x), **not** JHipster 9.x. npm's latest tag is actually 9.2.0 (Spring Boot 4.0.2, Jackson 3, Hibernate 7, Java 21 minimum, Node 22+) — a very recent, major cross-stack jump. Checked directly against the one dependency this whole project hinges on: **Timefold Solver does not have stable Spring Boot 4 support yet** — their tracking issue ([TimefoldAI/timefold-solver#1941](#)) is still open, targeted at a 2.0.0-beta line, and the newest published Spring Boot starter artifact on Maven Central is an -rc build, not GA. Pairing a “production-ready” requirement with a beta/RC solver build is the wrong tradeoff, so we deliberately stay one major version behind on JHipster until Timefold ships GA Spring Boot 4 support, then revisit.
 - **Java 21 LTS, pinned explicitly** (not whatever's newest on the machine — JHipster 8.11.0/Spring Boot 3.x is built and tested against Java 21/22, not the very latest JDK feature releases). Use a version manager (sdkman/jenv) to pin the project to 21 regardless of the system default.
 - **Node:** JHipster 8.11.0 requires `^18.19.0 || >=20.6.1` — notably *older/looser* than 9.x's `^22.18.0 || >=24.11.0`, so this also avoids forcing a Node upgrade just to run the generator.
 - **Maven** (better first-class Timefold/plugin ecosystem support than Gradle here)
 - **Application type:** monolith, **backend-only** (`--skip-client`) — this is explicitly “the backend”; no Angular/React/Vue scaffold generated
 - **Database:** PostgreSQL for **both dev and prod** (via Docker Compose in dev) — avoids H2/Postgres behavior drift, since we'll rely on Postgres-specific indexing/constraints
 - **Authentication type:** oauth2, but customized — see Section 10, Google Authentication Architecture
 - **Build additions:** `timefold-solver-spring-boot-starter` + `timefold-solver-test`, `openhtmltopdf-core` + `openhtmltopdf-pdfbox` (Apache-2.0 licensed HTML->PDF, no iText licensing complexity) for PDF export, `shedlock-spring` + `shedlock-provider-jdbc-template` (prevents the monthly cron and reminder jobs from double-firing if the app ever runs >1 instance)
 - **Testing:** JUnit 5 + Testcontainers (Postgres) — standard JHipster stack, reused as-is
-

2. Domain Model — Entities and Why

Evaluated every entity the original spec listed; a few are deliberately **collapsed** to avoid overengineering, with the reasoning stated inline.

User (reuse JHipster's OAuth2 User, don't duplicate)

JHipster's OAuth2-mode User entity already has `id` (String — populated from the IdP's sub claim), `login`, `firstName`, `lastName`, `email`, `langKey`, `activated` (boolean), plus JHipster audit fields. This directly satisfies the “extend rather than duplicate” requirement — **id IS the stable Google subject**, no separate `googleSubject` column needed, and `activated` is reused directly as the active/inactive flag instead of adding a duplicate field.

Added directly on User: - `role` — @ManyToOne to our own `Role` entity (nullable only transiently before bootstrap logic runs) - `canModerate`, `canPreach` — booleans, **not** a separate `ServiceCapability` entity/join table. Reasoning: it's a small, fixed, closed set (2 values today), read on every solver run and every eligibility check — a flat indexed boolean avoids a join for something queried constantly. The API DTO still exposes this as `capabilities`: `["CAN_MODERATE", "CAN_PREACH"]` so callers aren't coupled to storage; if a 3rd capability type is ever needed, a real join table can be introduced later without changing the contract.

We **drop** JHipster's default `Authority/jhi_user_authority` many-to-many — it's designed for static Spring Security `ROLE_*` strings, not our dynamic permission graph. See Section 9, Dynamic RBAC Design, for how `GrantedAuthoritys` are computed instead.

Role, Permission

- Role (id, name unique, description, audit fields) — many-to-many to Permission via role_permission
- Permission (id, code unique, description) — bulk-loaded idempotently from src/main/resources/config/permissions at startup (upsert by code; DB is source of truth after)

WeeklyPrayerConfiguration (the recurring template)

- WeeklyPrayerConfiguration (id, effectiveFrom, effectiveTo nullable, active)
- WeeklyPrayerConfigurationDay (id, configuration FK, dayOfWeek enum, requiresPreacher boolean) — exactly 7 rows per version, unique(configuration_id, day_of_week)

Changing the pattern **creates a new version** (new WeeklyPrayerConfiguration row with effectiveFrom = the change date) rather than mutating rows in place. The previous version's effectiveTo is closed off. Session generation picks whichever version's range covers the session's date — this is what makes historical rosters immune to later config changes.

PrayerSession (generated, actual dated data)

id, date (unique), dayOfWeek, requiresModerator, requiresPreacher, roster FK, requiresRescheduling (bool). requiresModerator/requiresPreacher are **snapshotted** from the config version active at generation time — so a later config change never retroactively alters an already-generated session.

PrayerAssignment — the Timefold planning entity (key design decision)

The spec explicitly asked to evaluate fields-on-PrayerSession vs a separate planning entity, and explain the choice. **Decision: separate PrayerAssignment entity.**

id, session FK, role (MODERATOR|PREACHER), user (nullable FK — the @PlanningVariable), locked (bool), generation FK, unique(session_id, role)

Why this beats two nullable fields directly on PrayerSession: 1. Fairness/consecutive-day/weekly-preach-limit constraints all reduce to one uniform stream over “assignments” instead of two parallel streams (one per field) that then need manually unioning. 2. It generalizes if a role is ever added later (e.g. musician) — just another PrayerAssignment row, no planning-entity shape change. 3. Calendar/PDF/history reads want exactly “list of (user, date, role)” — that's already the entity, no pivoting. 4. **Pinning** for rescheduling (below) is natural per-row via Timefold's @PlanningPin.

A bonus of this model: **PrayerAssignment rows are only created when required** — a moderation-only day never gets a PREACHER row at all. That means the hard constraint “no preacher on moderation-only days” is **structurally guaranteed by generation**, not something the solver needs to enforce — one less runtime constraint, and it's impossible to violate by construction.

UserAvailability

id, user FK, startDate, endDate, reason (nullable), status (ACTIVE|CANCELLED), audit fields. Validated at the service layer: startDate <= endDate, no overlapping ACTIVE ranges per user. (A Postgres EXCLUDE USING gist constraint could harden this further later; not needed for correctness at Phase 1.)

Roster vs RosterGeneration (the “evaluate both” ambiguity, resolved)

- **Roster** = the mutable lifecycle container for a planning period: id, periodFrom, periodTo, status (DRAFT|PUBLISHED|REQUIRES_RESCHEDULING|ARCHIVED), publishedAt. Owns the PrayerSessions in its date range.

- **RosterGeneration** = an **immutable, append-only audit row per solver invocation** against a Roster: id, roster FK, triggeredAt, triggeredBy, trigger (SCHEDULED_CRON|MANUAL|RESCHEDULE), planningFrom, planningTo, solverDurationMs, hardScore, softScore, feasible, status (RUNNING|COMPLETED|FAILED|INFEASIBLE), regenerated, rescheduleReason, errorMessage.

Every solve (initial generation or a reschedule) writes a new RosterGeneration row — this is what gives full audit history without ever needing to overwrite or lose a past run.

Notification, NotificationPreference, ReminderConfiguration, ReminderSent

- **Notification** — single entity, in-app record **and** email side-effect tracked on the same row: recipient FK, type enum, messageKey + JSON params (resolved to text at read time, in the user's locale — never pre-rendered, so it stays correct even if locale changes), relatedSession/relatedAssignment FK nullable, read (bool), readAt, emailStatus (PENDING|SENT|FAILED), emailSentAt, retryCount. One table instead of a separate email-log table — kept deliberately simple.
- **NotificationPreference** (1:1 per user) — just channel opt-in (emailEnabled).
- **ReminderConfiguration** — **global, admin-managed** list of offsets (daysBefore, active), seeded [7, 1]. *Assumption:* reminder timing reads as system-wide configurable policy, not per-user, so it's one shared admin-editable list rather than per-user settings.
- **ReminderSent** (assignment FK, reminderOffset FK, sentAt, **unique(assignment_id, reminder_offset_id)**) — the dedup ledger. The reminder job does insert-or-skip against this unique constraint inside the same transaction as sending, which combined with ShedLock on the job itself makes reminders idempotent even under concurrent/duplicate firing.

3. Entity Relationships (cardinalities, constraints)

User (1) — (1) Role — (M:M via role_permission) — Permission
 User (1) — (M) UserAvailability
 User (1) — (1) NotificationPreference
 User (1) — (M) Notification [recipient]

WeeklyPrayerConfiguration (1) — (7, exactly) WeeklyPrayerConfigurationDay unique(config_id, day)

Roster (1) — (M) PrayerSession PrayerSession.date is globally unique
 Roster (1) — (M) RosterGeneration
 PrayerSession (1) — (1 or 2) PrayerAssignment unique(session_id, role)
 PrayerAssignment (M) — (0..1) User [planning variable, nullable pre-solve]

ReminderConfiguration (1) — (M) ReminderSent — (M:1) PrayerAssignment unique(assignment_id, re

Audit fields (createdAt/By, updatedAt/By) reuse JHipster's existing AbstractAuditingEntity base class on every entity above rather than hand-rolling them. Key indexes: User.id (already PK = Google sub), Role.name unique, Permission.code unique, PrayerSession.date unique, PrayerAssignment(session_id, role) unique, UserAvailability(user_id, startDate, endDate) composite, Notification(recipient_id, read, createdAt) composite.

4–5. Weekly Configuration vs Generated Sessions

Kept structurally separate: WeeklyPrayerConfiguration[Day] is the template, PrayerSession/PrayerAssignment are the generated planning data. Generation logic for a period [from,

to]: for each date in range, find the config version covering that date, read its dayOfWeek row, create a PrayerSession with snapshotted requiresModerator/requiresPreacher, then create 1 or 2 PrayerAssignment rows (MODERATOR always; PREACHER only if requiresPreacher).

6. Timefold Planning Model

```
@PlanningSolution RosterSolution
    @ProblemFactCollectionProperty List<User> eligibleUsers // active + (canModerate || canPreach)
    filtered
    @ProblemFactCollectionProperty List<PrayerSession> sessions
    @PlanningEntityCollectionProperty List<PrayerAssignment> assignments
    @PlanningScore HardSoftScore score

@PlanningEntity PrayerAssignment
    @PlanningVariable(valueRangeProviderRefs = "userRange") User user
    @PlanningPin boolean locked // true for unaffected assignments on a published roster during reschedule
```

- **Value range:** a single broad range (all active eligible users), with capability matching enforced via hard constraints rather than a role-filtered value range. At this scale (dozens of users, ~30–60 sessions/month) solver efficiency from filtered ranges is a non-issue; simplicity wins for Phase 1.
- **Inactive users:** filtered out before the solve even starts (never enter eligibleUsers), *plus* kept as a defensive hard constraint in case of stale data.
- **Unavailability:** cannot be pre-filtered globally since it's per-date — implemented as a real constraint-stream join between assignment->user->UserAvailability and session.date.
- **Rescheduling minimization:** implemented via **@PlanningPin**, not a soft penalty constraint. Unaffected assignments on a published roster are pinned (Timefold structurally cannot move them), which is a *hard guarantee* stronger than a soft penalty — so a “minimize changes to published roster” constraint is **not needed as a runtime constraint**; pinning already fully satisfies “don’t unnecessarily reshuffle unaffected assignments.” Only sessions flagged requiresRescheduling are unpinned for a reschedule solve.

7. Hard Constraints (Constraint Streams, HardSoftScore)

Constraint	Logic
everyAssignmentMustBeFilled	assignment.user == null -> penalize. Covers both “moderator required” and “preacher required when configured”, since rows only exist when required.
moderatorAndPreacherMustDiffer	same session, MODERATOR.user == PREACHER.user (both non-null) -> penalize
preacherAtMostOncePerIsoWeek	group by (user, role=PREACHER, ISO week of session.date), penalize any group with count > 1
inactiveUserCannotBeAssigned	defensive re-check even though pre-filtered
unavailableUserCannotBeAssigned	join to UserAvailability (status=ACTIVE) covering session.date
userMustHaveModerationCapability	role=MODERATOR && !user.canModerate
userMustHavePreachingCapability	role=PREACHER && !user.canPreach

“No preacher on moderation-only days” is **structurally guaranteed** by generation (see Section 2, PrayerAssignment — the Timefold planning entity), documented but not implemented as a runtime constraint.

8. Soft Constraints

Constraint	Logic
balanceModerationAssignments	loadBalance over users with canModerate=true
balancePreachingAssignments	loadBalance over users with canPreach=true
avoidConsecutiveAssignments	penalize same user assigned (any role) on two consecutive dates
minimizeAssignmentImbalance	overall loadBalance across total assignment count, lower weight than the two role-specific ones — catches gross imbalance across the whole roster

Fairness is naturally eligibility-scoped since the value range only ever contains active, capability-matching users — a moderate-only user is never compared against preaching load. **Known simplification:** fairness balances by raw count among eligible users, not normalized by each user's availability-adjusted capacity within the period (someone available 1 week of 4 isn't held to the same count as someone available all 4). Flagging this now rather than silently deciding it — full availability-weighted fairness is a reasonable later refinement, not needed for MVP correctness. Relative constraint weights get tuned empirically once real data exists.

9. Dynamic RBAC Design (ADMIN vs SUPER_ADMIN)

Two layers, kept deliberately separate to avoid colliding with JHipster's built-in admin console:

1. **Business authorization** (our domain): Role (seeded SUPER_ADMIN, ADMIN, USER) -> Permission (from permissions.json). On each authenticated request, a custom converter computes GrantedAuthoritys dynamically as PERM_<code> for every permission the user's role holds — never a hardcoded role check. @PreAuthorize("hasAuthority('PERM_ROSTER_GENERATE')") everywhere. A short-lived cache (Caffeine, ~60s TTL, keyed by user id) avoids a DB round-trip per request while still reflecting permission changes promptly.
2. **JHipster's own infrastructure gate:** JHipster's generated SecurityConfiguration protects /management/** (actuator: health, metrics, loggers) with the **static** ROLE_ADMIN authority. We keep that mechanism *only* for this narrow purpose — it is **not** our business ADMIN domain role. The converter additionally grants the literal ROLE_ADMIN authority **only** to users whose domain Role.name == SUPER_ADMIN. Business ADMIN gets PERM_* authorities from its assigned permissions and nothing from JHipster's static role at all.

This means “ADMIN” as a word means two unrelated things in the codebase, deliberately: our seeded domain Role row named ADMIN (fully dynamic, editable, permission-driven), and Spring's static ROLE_ADMIN string (used solely to gate JHipster's actuator console, granted only to SUPER_ADMIN). We drop JHipster's default Authority/jhi_user_authority persistence entirely rather than trying to reuse it for both purposes.

10. Google Authentication Architecture

- spring.security.oauth2.resourceserver.jwt.issuer-uri: https://accounts.google.com — Spring auto-validates issuer, signature (via Google's JWKS), and expiry.
- **Required addition:** a custom OAuth2TokenValidator<Jwt> checking the aud claim equals GOOGLE_CLIENT_ID, combined via DelegatingOAuth2TokenValidator with Spring's defaults — issuer-uri alone does **not** validate audience, and skipping this is a real security gap for Google-as-resource-server setups.

- Frontend is expected to use Google Identity Services (client-side “Sign in with Google”) to obtain an **ID token** (a JWT) and send it as `Authorization: Bearer <id_token>` — not the opaque access token, which Google doesn’t issue as a JWT. `GOOGLE_CLIENT_SECRET` likely isn’t needed server-side under this pattern since we never perform the code exchange ourselves; flagging as an assumption to confirm once the frontend’s actual flow is known.
- On each validated request, provision-or-load the local User by `id = sub claim`; assign `Role=USER` by default.

Super Admin bootstrap

`INITIAL_SUPER_ADMIN_EMAIL` env var. On **first-ever creation** of a User row (not on every login — this matters: if `SUPER_ADMIN` is later reassigned away from this account, subsequent logins must not silently re-grant it), if email case-insensitively matches the configured value, assign `Role=SUPER_ADMIN`; otherwise `Role=USER`. No public self-promotion endpoint exists anywhere.

11. Roster Lifecycle

DRAFT → PUBLISHED → (REQUIRES_RESCHEDULING ↔ PUBLISHED) → ARCHIVED

Generation (cron, manual, or reschedule) **auto-publishes on success** — DRAFT is real but transient, used internally during the generate→solve→validate pipeline; it only becomes a terminal state if the solve fails or is infeasible (nothing publishes in that case, `RosterGeneration.status=FAILED/INFEASIBLE` and the Roster stays DRAFT for an admin to investigate/retry). ARCHIVED is set once a period is fully in the past (a lightweight scheduled sweep, or lazily on read).

12. Rescheduling Architecture

- **Detection is event-driven and immediate:** submitting `UserAvailability`, deactivating a user, or changing the weekly config publishes a domain event; a listener finds affected future `PrayerAssignments` (assigned user + session date falls in the new unavailable range / user now inactive) and flags their `PrayerSession.requiresRescheduling = true`, moving the owning Roster to `REQUIRES_RESCHEDULING`.
 - **Re-solve then auto-publishes:** a solve runs automatically against a fresh `RosterSolution` scoped to the affected Roster, where every `PrayerAssignment` on a session *not* flagged `requiresRescheduling` is `@PlanningPin`-locked. Only flagged sessions’ assignments are free variables. All hard constraints are re-validated; on success the Roster returns to `PUBLISHED`, a new `RosterGeneration(trigger=RESCHEDULE)` row is written, and both the removed and newly-assigned users are notified.
 - `POST /api/rosters/{id}/reschedule` still exists as a manual “force it now” endpoint — useful if the automatic listener is debounced/async and an admin doesn’t want to wait, or to retry after a failed auto-reschedule.
 - Past sessions (`date < today`) are never touched by any reschedule pass.
-

13. Notification Architecture

`NotificationService` (writes the `Notification` row + publishes a `NotificationCreatedEvent`) → `async @EventListener EmailNotificationService` (renders a localized Thymeleaf template, sends, updates `emailStatus/retryCount` on the same row). Roster generation/publish never blocks on email

— it's fire-and-forget via Spring events, so a mail provider outage never fails a roster publish. A scheduled retry sweep resends PENDING/FAILED rows up to a small max retry count.

ReminderService — a @Scheduled (ShedLock-guarded) daily job: for each active ReminderConfiguration offset, find assignments where session.date == today + offsetDays, insert into ReminderSent (unique constraint = the idempotency guarantee) and send only if the insert succeeded.

All notification/email text lives in messages_fr.properties (default) / messages_en.properties (future) as message keys with params — never hardcoded strings in Java.

PDF export: GET /api/rosters/{id}/pdf (permission ROSTER_VIEW) renders a Thymeleaf HTML template (clean typography, week-grouped table, accent header, alternating row shading) to PDF via openhtmltopdf. Reuses the same Thymeleaf skill already needed for email templates. A personal GET /api/me/prayer-assignments/pdf calendar export is a natural follow-on using the same mechanism.

14. API Design

```
GET    /api/prayer-config/weekly/history    # past configuration versions (auditability)
GET    /api/rosters/{id}/generations      # RosterGeneration audit trail for a roster
GET    /api/rosters/{id}/pdf              # PDF export
GET    /api/me/prayer-assignments/pdf     # personal calendar PDF
POST   /api/rosters/{id}/reschedule      # manual "force reschedule now"
```

(plus the full CRUD/roster/availability/notification endpoints from the original API design.)

All entities exposed via DTOs (MapStruct), never raw JPA entities. Errors use Spring's ProblemDetail with structured error keys (JHipster's existing ExceptionTranslator pattern extended), French default messages. Infeasible solves return 422 with a diagnostic breakdown (which sessions/roles have no feasible candidate) built from Timefold's SolutionManager.analyze() / score explanation API.

15. Package/Module Structure (modular monolith)

```
com.<org>.prayerroster
├─ user          (User extension, capabilities, availability)
├─ authorization (Role, Permission, RBAC converter, security config, SUPER_ADMIN bootstrap)
├─ prayerconfig  (WeeklyPrayerConfiguration[Day])
├─ roster        (Roster, RosterGeneration, PrayerSession, PrayerAssignment, lifecycle, rescheduling)
├─ scheduling    (Timefold: RosterSolution, ConstraintProvider, SolverService)
├─ notification  (Notification, NotificationPreference, ReminderConfiguration/Sent, Email/InApp)
└─ shared        (JHipster's AbstractAuditingEntity reuse, error handling, i18n helpers)
```

Each module follows JHipster's per-feature convention internally (domain, repository, service, service/dto, service/mapper, web/rest).

Assumptions flagged for review (safe defaults chosen, not yet confirmed)

1. Reminder offsets ([7, 1] days) are **global, admin-configured policy**, not per-user settings.
2. Google CLIENT_SECRET may end up unused server-side under the resource-server pattern — to confirm once the actual frontend flow is built.
3. Fairness is balanced by raw assignment count among eligible users, not normalized by each user's availability-adjusted capacity in the period — a possible future refinement, not MVP scope.

Next Steps

Phase 2 (JHipster project setup: exact `jhipster` CLI invocation/config for backend-only + oauth2 + postgresql) begins once this document is approved, followed by domain/Liquibase implementation, security, Timefold, roster/rescheduling, notifications, REST API, tests, and Docker/deployment.